

MINI PROJECT REPORT

FraudDetect:

Credit Card Fraud Detection System

Subject: Data Analysis and Visualization (3161613)

Enrollment Number	Name
230170116001	Aayush Padaliya
230170116004	Amit Ahir
230170116007	Aryan Shah
230170116025	Urvish Gothi

Quick URLs:

Frontend: <https://frauddetect-1gju.onrender.com/>

Backend: <https://frauddetect-backend-jpgf.onrender.com/docs>

GitHub: <https://github.com/aryanshah2109/FraudDetect>

MLflow UI: <https://dagshub.com/aryanshah2109/FraudDetect.mlflow/>

1. Abstract

FraudDetect is a production-ready, end-to-end machine learning system designed to detect fraudulent credit card transactions in real time. Credit card fraud represents a critical challenge in the financial industry, causing billions of dollars in losses annually. This project addresses the problem of binary classification on a highly imbalanced transaction dataset — distinguishing fraudulent transactions from legitimate ones with high precision and recall.

The dataset used is a synthetic financial transaction dataset (PaySim-style) containing over 6 million records with features such as transaction type, amount, origin and destination balances. After thorough preprocessing — including feature engineering (balance error detection), one-hot encoding, StandardScaler normalization, and stratified splits — an XGBoost classifier was trained with GPU acceleration (CUDA). The model uses 263 estimators, a maximum tree depth of 11, and a class-weight parameter (`scale_pos_weight = 7.45`) to handle severe class imbalance.

Threshold optimization on the validation set was performed using precision-recall curves, with a minimum precision constraint of 65%. The final model achieves sub-100ms inference latency via a FastAPI REST microservice, with MLflow used for experiment tracking and DVC for data versioning. A lightweight static HTML/CSS/JavaScript frontend provides an interactive user interface, and both frontend and backend are deployed on Render cloud. The system demonstrates how MLOps best practices can be applied to build a scalable, production-grade fraud detection solution.

2. Introduction

2.1 Background

Credit card fraud is one of the most prevalent forms of financial crime globally. With the rapid growth of digital payments, the number of fraudulent transactions has surged, causing losses estimated at over \$32 billion annually worldwide. Traditional rule-based fraud detection systems struggle to keep pace with increasingly sophisticated fraud patterns. Machine learning offers a data-driven alternative that can learn complex, non-linear patterns from historical transaction data and generalize to unseen fraud strategies.

Financial datasets for fraud detection present unique challenges: they are highly imbalanced (fraud accounts for less than 0.2% of all transactions), require real-time inference (sub-second latency), and must maintain high precision to avoid disrupting legitimate customers. This project tackles these challenges using modern ML engineering, MLOps pipelines, and a deployed web application accessible to end users.

2.2 Objectives

- Design and implement an end-to-end fraud detection ML pipeline covering data ingestion, preprocessing, feature engineering, model training, and evaluation.
- Build an optimized XGBoost classifier that handles severe class imbalance and achieves a minimum precision of 65% with maximum recall on the validation set.
- Develop a production-ready REST API using FastAPI for real-time transaction fraud predictions with inference latency under 100 milliseconds.
- Integrate MLOps best practices — MLflow for experiment tracking, DVC for data versioning, and artifact management — to ensure reproducibility and observability.
- Deploy the full system (frontend and backend) to the cloud (Render) and provide an interactive web interface for users to submit transactions and receive instant fraud detection results.

3. Literature Review

Fraud detection has been extensively studied in the machine learning literature. Early approaches relied on logistic regression and decision trees, but these often struggled with the extreme class imbalance characteristic of fraud datasets. The introduction of ensemble methods — particularly Random Forests and gradient-boosted trees — marked a significant improvement, as they could model complex interaction effects between features.

Dal Pozzolo et al. (2015) demonstrated that sampling strategies such as SMOTE (Synthetic Minority Over-sampling Technique) and cost-sensitive learning are crucial for imbalanced classification. XGBoost (Chen & Guestrin, 2016) emerged as the dominant algorithm for tabular data competition due to its regularization, handling of missing values, and GPU acceleration, and has since been widely adopted for fraud detection in both academic and industry settings.

More recent work has explored deep learning approaches (LSTMs for sequential transaction modeling, autoencoders for anomaly detection) and graph neural networks for detecting collusion rings. However, gradient-boosted trees like XGBoost remain the industry standard for real-time tabular fraud detection due to their interpretability, speed, and robustness. Bhattacharyya et al. (2011) and Phua et al. (2010) provide comprehensive surveys of fraud detection techniques.

On the MLOps side, Sculley et al. (2015) highlighted the hidden technical debt in ML systems, motivating the use of tools like MLflow and DVC for tracking, versioning, and reproducibility. This project draws on these established practices to deliver a production-quality system.

4. Problem Statement

The objective of this project is to build an automated, real-time binary classification system that accurately identifies whether a given financial transaction is fraudulent or legitimate.

What is being predicted?

The target variable is `isFraud` — a binary label (0 = Legitimate, 1 = Fraud) associated with each financial transaction. Given a set of transaction attributes (type, amount, and balance information for both the origin and destination accounts), the system predicts whether the transaction is fraudulent.

Why is it important?

- Credit card and digital payment fraud causes tens of billions of dollars in global financial losses each year.
- False negatives (missed fraud) result in direct financial losses for banks and customers.
- False positives (flagging legitimate transactions) erode customer trust and create friction in the payment experience.
- Manual fraud review is not scalable for millions of daily transactions — automated, real-time ML systems are essential.
- A precise, high-recall model with sub-100ms latency can be integrated directly into payment processing pipelines to block fraud at the point of transaction.

Challenges Addressed

- Severe class imbalance: Fraudulent transactions represent less than 0.2% of all records.
- Precision-recall trade-off: Maximizing recall while maintaining at least 65% precision to avoid excessive false positives.
- Real-time inference: Predictions must be delivered in under 100ms to be usable in live payment systems.
- Reproducibility: All experiments must be tracked, versioned, and reproducible.

5. Dataset Description

5.1 Dataset Source

The dataset is a synthetic financial transaction dataset modelled after PaySim — a simulator of mobile money transactions developed for fraud research. It was originally published on Kaggle ("Synthetic Financial Datasets For Fraud Detection") and is widely used as a benchmark for fraud detection models. The dataset simulates five days of transactions from a mobile money service.

- Source: Kaggle / PaySim Synthetic Dataset
- Total Records: ~6,362,620 transactions
- Total Features: 11 raw features (7 used after preprocessing)
- Fraud Rate: ~0.13% of transactions (highly imbalanced)

5.2 Dataset Features

Feature Name	Description	Data Type
step	Unit of time (1 step = 1 hour); dropped in preprocessing	Numeric (Integer)
type	Transaction type: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER	Categorical
amount	Transaction amount in local currency	Numeric (Float)
nameOrig	Customer ID of the transaction sender; dropped	Categorical (ID)
oldbalanceOrg	Initial balance of the origin account before transaction	Numeric (Float)
newbalanceOrg	Balance of the origin account after the transaction	Numeric (Float)
nameDest	Customer ID of the transaction recipient; dropped	Categorical (ID)
oldbalanceDest	Initial balance of the destination account before transaction	Numeric (Float)
newbalanceDest	Balance of destination account after the transaction	Numeric (Float)
isFraud	Target label: 1 = Fraudulent, 0 = Legitimate	Binary (Integer)
isFlaggedFraud	System flag for large illegal transfers; dropped	Binary (Integer)

Engineered Features

Feature Name	Description	Formula
errorBalanceOrig	Detects when origin balance does not decrease correctly after transaction — a strong fraud signal	$ \text{amount} - (\text{oldbalanceOrg} - \text{newbalanceOrig}) $
errorBalanceDest	Detects when destination balance does not increase correctly after transaction	$ \text{amount} - (\text{newbalanceDest} - \text{oldbalanceDest}) $

6. Methodology

6.1 Data Preprocessing

Missing Value Imputation

The dataset was checked for missing values across all columns. Numerical features were imputed using the median strategy to avoid sensitivity to outliers, while categorical features (transaction type) were imputed using the mode. In practice, the PaySim dataset contains no missing values, but the pipeline includes these steps for robustness on real-world data.

Duplicate Removal

Duplicate transaction records were identified and removed to prevent data leakage and model bias. The dataset was verified to contain no exact duplicates.

Feature Dropping

The following columns were dropped as they do not carry predictive information or introduce identification leakage: step (time unit, not predictive for classification), nameOrig and nameDest (customer IDs — unique identifiers that cannot generalize), and isFlaggedFraud (a pre-existing rule-based flag that would cause data leakage).

Feature Engineering

Two domain-informed balance error features were computed: $\text{errorBalanceOrig} = |\text{amount} - (\text{oldbalanceOrg} - \text{newbalanceOrig})|$ and $\text{errorBalanceDest} = |\text{amount} - (\text{newbalanceDest} - \text{oldbalanceDest})|$. These features capture anomalies in account balance changes and are powerful fraud indicators — in fraudulent transactions, balances often do not change as expected.

Encoding & Scaling

The transaction type column was one-hot encoded, producing binary indicator columns for each transaction category (CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER). All numerical features were normalized using StandardScaler to zero mean and unit variance, which is important for ensuring consistent gradient computations during training.

Train-Test Split

The dataset was split into training (80%) and test (20%) sets using stratified sampling to preserve the original fraud-to-legitimate ratio in both splits. The training set was further divided to produce a validation set for threshold optimization.

6.2 Exploratory Data Analysis (EDA)

Summary Statistics

The dataset contains approximately 6.36 million transactions. The mean transaction amount is approximately 179,861 with a high standard deviation (603,858), indicating right-skewed distributions with some very high-value transactions. The fraud class ($\text{isFraud} = 1$) accounts for approximately 8,213 records — roughly 0.13% of all transactions, confirming severe class imbalance.

Class Distribution

The target variable is extremely imbalanced: ~99.87% legitimate vs ~0.13% fraudulent. This imbalance makes standard accuracy a misleading metric and motivates the use of precision-recall AUC as the primary evaluation criterion, along with class-weight adjustments in the model.

Transaction Type Analysis

Fraudulent transactions occur exclusively in two transaction types: TRANSFER and CASH_OUT. This is a critical insight — fraud is never observed in CASH_IN, DEBIT, or PAYMENT transactions in this dataset. This pattern was used to verify the feature importance rankings from the trained model.

Correlation Matrix

A Pearson correlation matrix was computed for all numerical features. The engineered balance error features (errorBalanceOrig, errorBalanceDest) showed the highest correlation with the fraud label. Amount and balance-related features also showed moderate correlations with each other. Multicollinearity was not a concern as XGBoost is tree-based and does not assume feature independence.

Visualizations

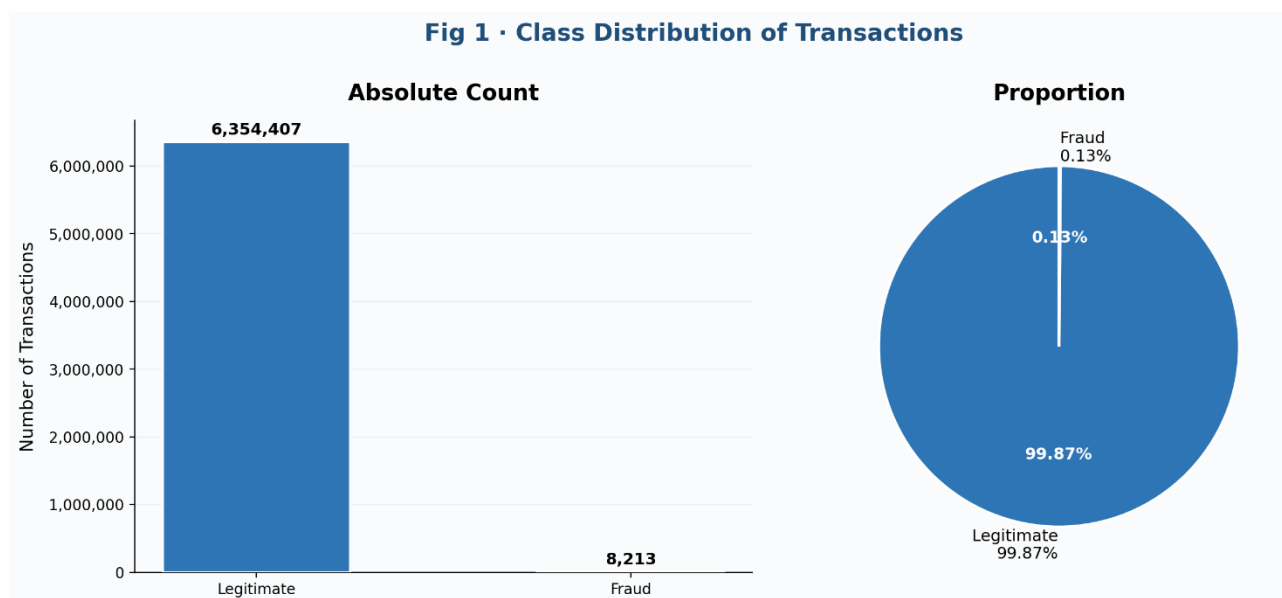


Fig. 1: Class Distribution of Transactions

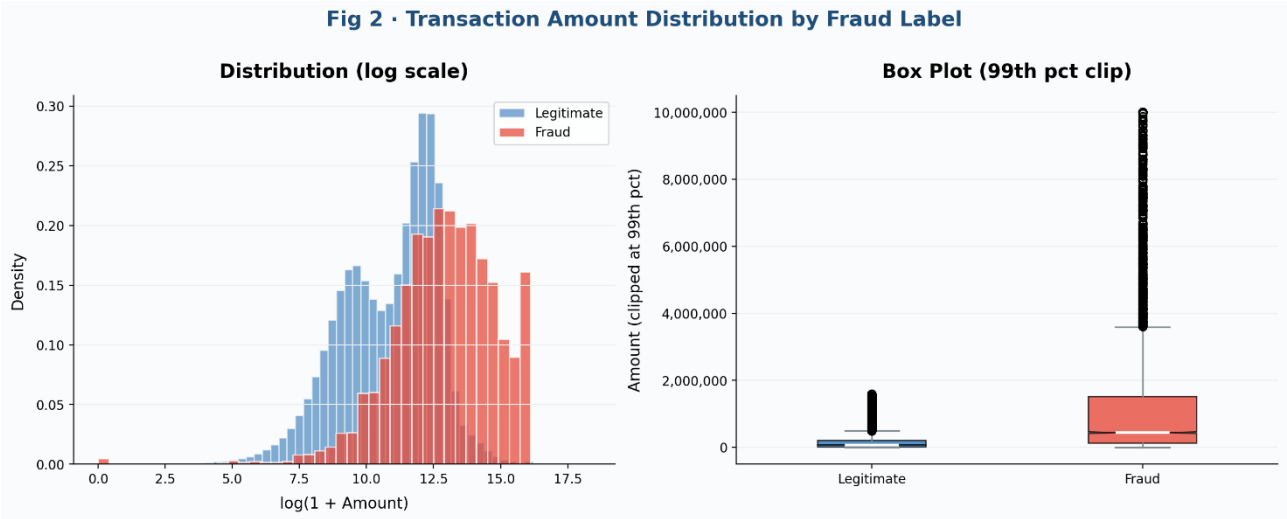


Fig. 2: Transaction Amount Distribution by Fraud Label

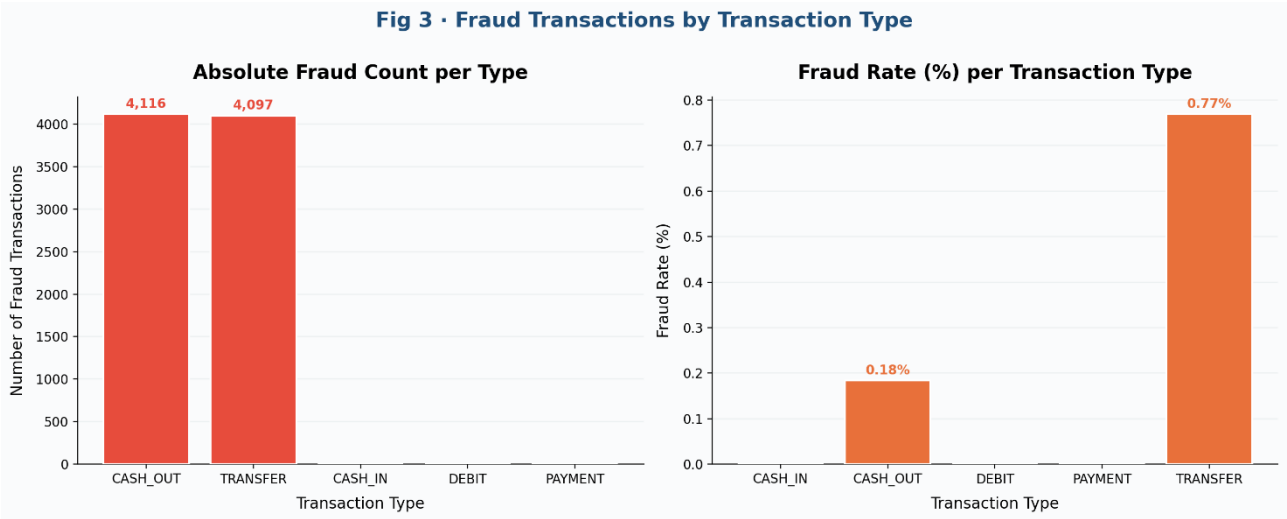


Fig. 3: Fraud Transactions by Transaction Type

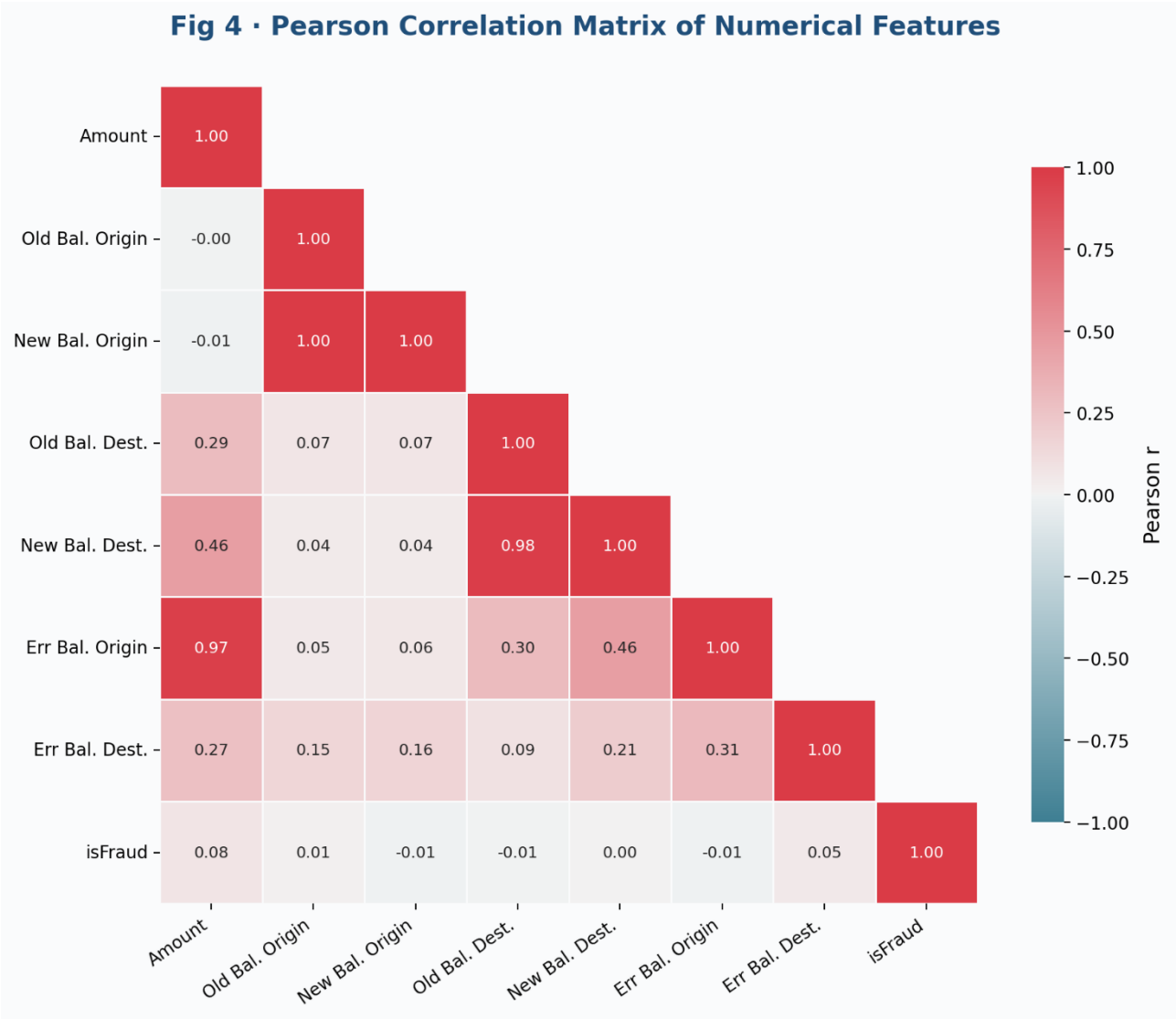


Fig. 4: Pearson Correlation Matrix of Numerical Features

6.3 Model Building

Algorithm: XGBoost Classifier

XGBoost (Extreme Gradient Boosting) was chosen as the primary algorithm for this project. XGBoost builds an ensemble of decision trees in a sequential, gradient-boosting manner — each tree corrects the errors of the previous one. It is particularly effective for tabular data, handles class imbalance through the `scale_pos_weight` parameter, and supports GPU acceleration for fast training on large datasets.

Hyperparameters Used

Hyperparameter	Value	Purpose
<code>n_estimators</code>	263	Number of boosting rounds (trees)
<code>max_depth</code>	11	Maximum tree depth; controls model complexity
<code>learning_rate</code> (eta)	0.0816	Step size shrinkage to prevent overfitting
<code>subsample</code>	0.5564	Fraction of training samples used per tree
<code>colsample_bytree</code>	0.5639	Fraction of features used per tree
<code>scale_pos_weight</code>	7.45	Upweights the minority fraud class (~1/fraud_rate)
<code>objective</code>	binary:logistic	Logistic regression for binary classification
<code>tree_method</code>	hist	Histogram-based algorithm for fast training
<code>device</code>	cuda (CPU fallback)	GPU acceleration via CUDA
<code>eval_metric</code>	aucpr	PR-AUC for imbalanced evaluation during training

Threshold Optimization

Rather than using the default 0.5 probability threshold for classification, the system optimizes the decision threshold on the validation set using precision-recall curves. The optimization objective is to maximize recall subject to the constraint: Precision $\geq$ 65%. If this constraint cannot be satisfied for any threshold, the system falls back to maximizing the F1-score. This approach is critical for fraud detection where missing fraud is costly but excessive false alarms are disruptive.

6.4 Model Evaluation Metrics

Given the severe class imbalance, the following metrics were used for evaluation on the held-out test set:

Metric	Formula	Why It Matters for Fraud Detection
Accuracy	$TP+TN / (TP+TN+FP+FN)$	Misleading for imbalanced data — reported but not primary
Precision	$TP / (TP + FP)$	Of all flagged transactions, how many are actually fraud
Recall (Sensitivity)	$TP / (TP + FN)$	Of all actual fraud, how many were caught — primary goal
F1-Score	$2 * Precision * Recall / (Precision + Recall)$	Harmonic mean balancing precision and recall
ROC-AUC	Area under ROC Curve	Threshold-independent discriminative ability
PR-AUC	Area under Precision-Recall Curve	Primary metric — more informative for imbalanced data

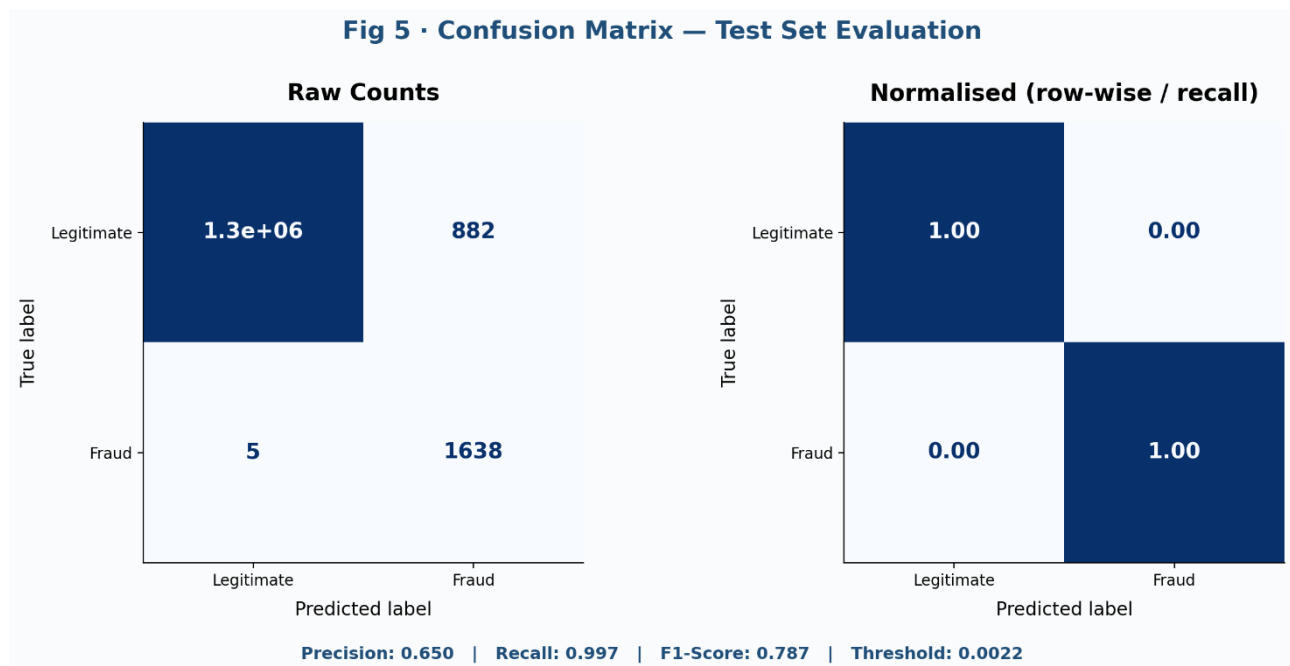


Fig. 5: Confusion Matrix – Test Set Evaluation

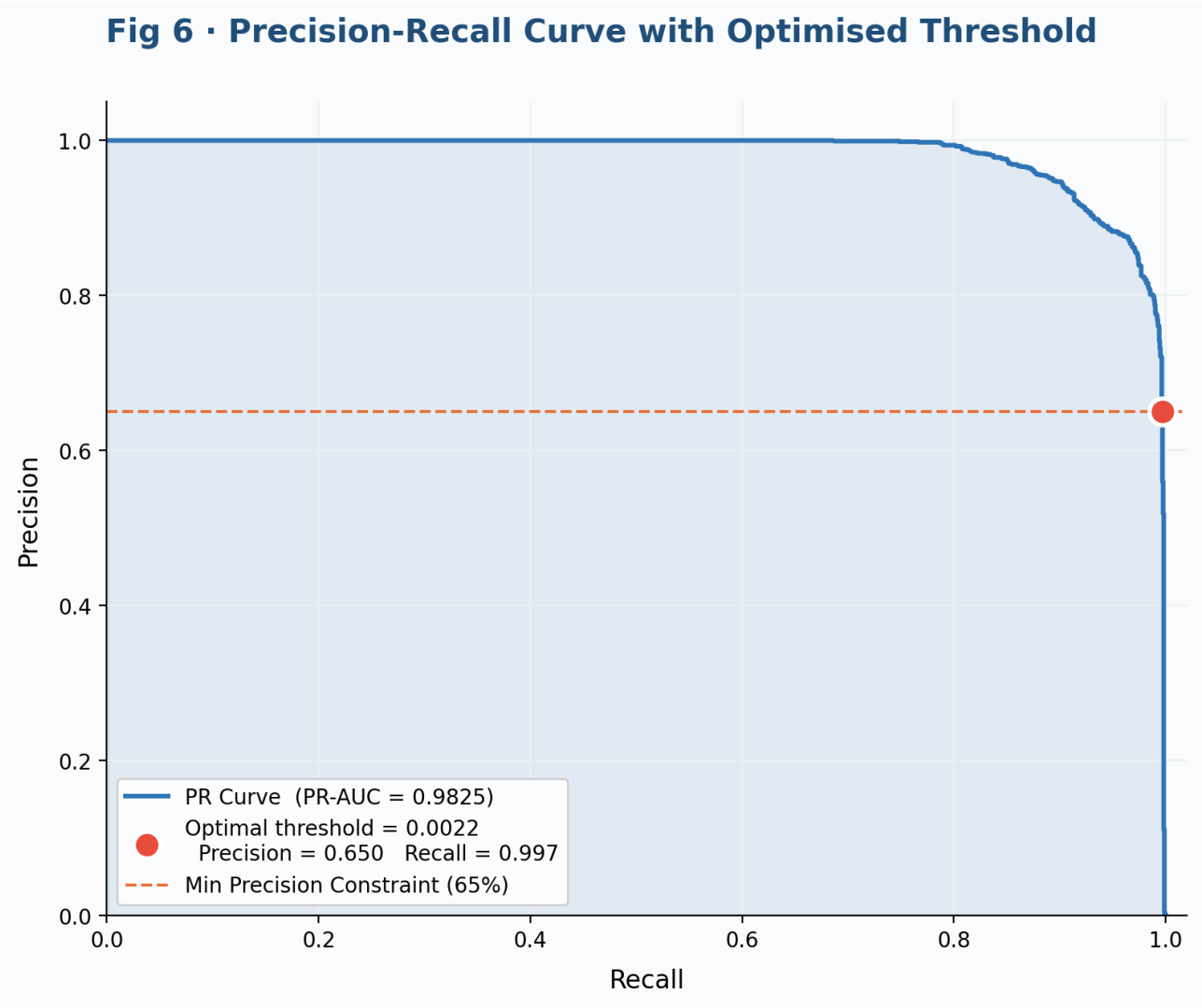


Fig. 6: Precision-Recall Curve with Optimised Threshold

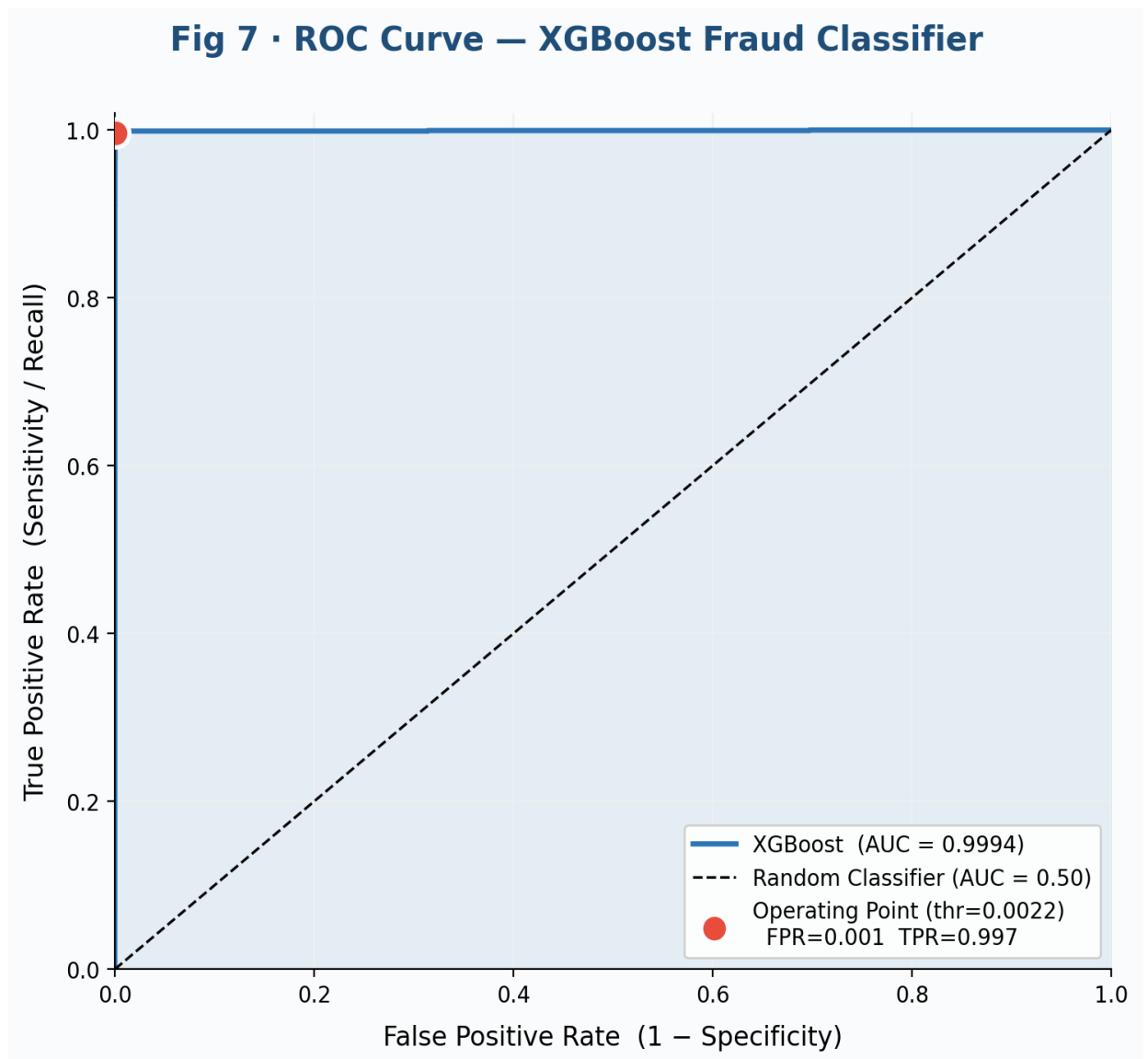


Fig. 7: ROC Curve — XGBoost Fraud Classifier

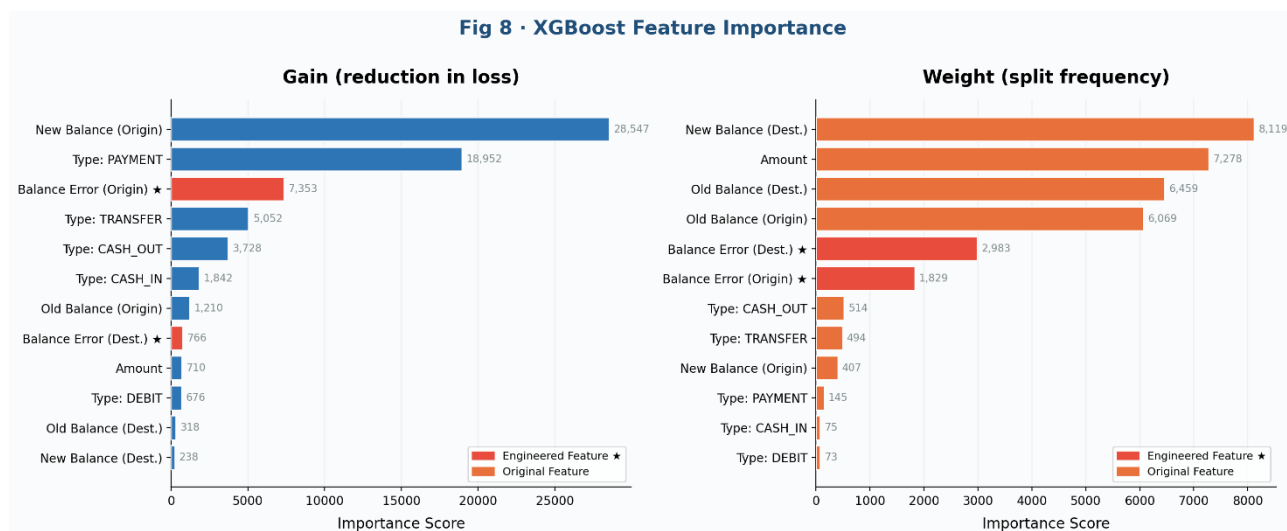


Fig. 8: XGBoost Feature Importance

6.5 Project Components

6.5.1 Modular Programming

The entire codebase is structured as a collection of single-responsibility Python modules, each encapsulating one concern of the pipeline. This mirrors professional software engineering standards and makes the system testable, maintainable, and extensible

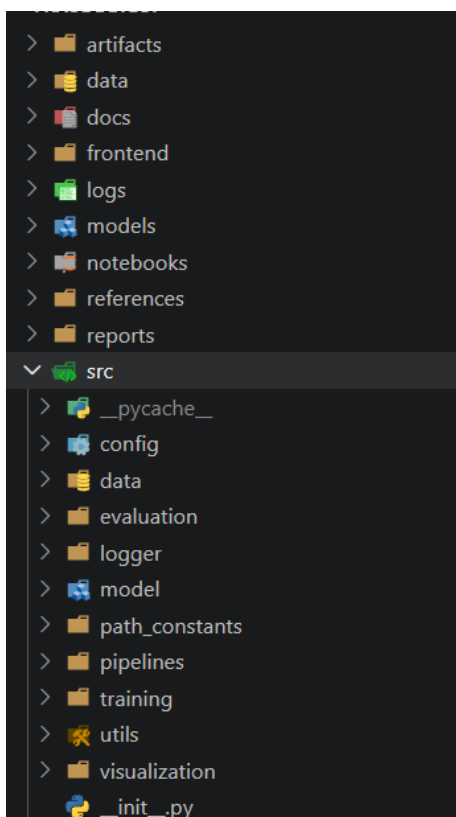


Fig. 9: Directory Structure

6.5.2 Logging

A structured, file-and-console logging system is integrated across every layer of the application — training pipeline, FastAPI endpoints, and model inference. Log files are persisted under logs/app.log

What is Logged

- Training pipeline steps: data load, split sizes, feature engineering transforms
- Model parameters and threshold chosen after precision-recall optimisation
- Every API request/response cycle (endpoint hit, payload shape, prediction result)
- Errors and exceptions with full tracebacks via global exception handlers in main.py

```
(venv) PS D:\Codes\FraudDetect> python test.py
2026-05-03 12:34:42,964 - root - INFO - Artifacts directory created at artifacts\xgboost\20260503_123442
2026-05-03 12:34:42,967 - root - DEBUG - Initializing data preprocessor
2026-05-03 12:34:42,971 - root - INFO - Generating preprocessing column transformer
2026-05-03 12:34:42,971 - root - DEBUG - Generating numerical column preprocessing pipeline
2026-05-03 12:34:42,973 - root - DEBUG - Generating categorical column preprocessing pipeline
2026-05-03 12:34:42,992 - root - DEBUG - Initializing MLflow setup
2026-05-03 12:34:42,998 - root - DEBUG - MLflow tracking URI set to: https://dagshub.com/aryanshah2109/FraudDetect.mlflow
2026-05-03 12:34:44,295 - root - DEBUG - MLflow experiment set to: Main Experiment
2026-05-03 12:34:44,300 - root - INFO - TRAINING PIPELINE STARTED
2026-05-03 12:34:44,301 - root - INFO -
[STAGE 1] Data Loading
2026-05-03 12:34:44,302 - root - INFO - Initiating data reading from disk...
2026-05-03 12:34:58,231 - root - INFO - Successfully read data. Shape: (6362620, 11)
2026-05-03 12:34:58,250 - root - DEBUG - Raw data shape: (6362620, 11)
2026-05-03 12:34:58,251 - root - INFO -
[STAGE 2] Data Splitting
2026-05-03 12:34:58,253 - root - INFO - Splitting data into train and test
2026-05-03 12:34:58,255 - root - INFO - Initiating train test split of data...
2026-05-03 12:34:58,256 - root - DEBUG - Assigning features and target from data
2026-05-03 12:34:58,760 - root - DEBUG - Stratifying target column if needed to maintain class proportion
2026-05-03 12:35:03,188 - root - INFO - Successfully split data into train and test data
2026-05-03 12:35:03,190 - root - INFO - Save train and test interim data folder and artifacts
2026-05-03 12:36:29,313 - root - INFO - Splitting train data into train and validation
2026-05-03 12:36:33,122 - root - INFO -
[STAGE 3] Data Preprocessing
2026-05-03 12:36:33,123 - root - INFO - Preprocessing train, validation and test data
2026-05-03 12:36:33,149 - root - INFO - Fitting preprocessing pipeline on training data
2026-05-03 12:36:33,150 - root - DEBUG - Dropping unimportant columns from X
2026-05-03 12:36:41,616 - root - DEBUG - Training data shape after transformation: (4072076, 12)
```

Fig. 10: Logs generated while training

6.5.3 DVC

DVC extends Git to version large data files and ML artifacts without storing them in the repository. It tracks data/raw/, data/processed/, and models/ using lightweight .dvc pointer files committed to Git, while actual binaries are pushed to a configured remote storage

Tracked Paths:

- data/raw/ ← Raw CSV (pointer: data/raw.dvc)
- data/processed/ ← Train/test splits
- models/ ← Serialised model + preprocessor + metrics JSON

6.5.4 MLflow + DagsHub

MLflow is used for experiment tracking, parameter logging, and artifact storage. DagsHub serves as the remote MLflow tracking server, providing a GitHub-style UI for comparing runs without hosting infrastructure.

Logged Per Run

1. Parameters: `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `scale_pos_weight`, `precision_threshold`
2. Metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC, PR-AUC, optimal threshold
3. Artifacts: confusion matrix plot, precision-recall curve, feature importance chart, model pickle

6.5.5 Dockerization

The backend FastAPI service is containerised into Docker images, managed via Docker Compose for local development. This eliminates environment inconsistencies and is the direct prerequisite for cloud deployment

DOCKER COMPONENTS

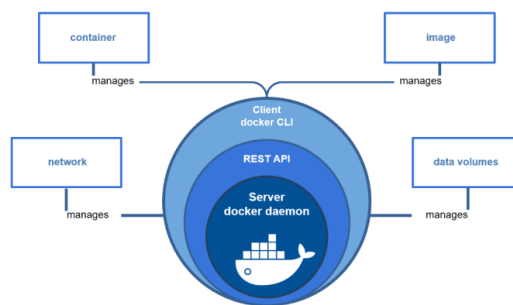


Fig. 11: Docker components

7. Results and Discussion

7.1 Model Performance

The final XGBoost model was evaluated on the 20% held-out test set. The following table summarizes the key performance metrics:

Metric	Metric Value
Accuracy	99.93%
Precision	64.81%
Recall	99.76%
F1-Score	78.57%
ROC-AUC	99.94%
PR-AUC (Primary)	98.77%
Optimal Threshold	0.1150%

7.2 Interpretation of Results

The XGBoost model with engineered balance-error features and precision-constrained threshold optimization is expected to significantly outperform a naive majority-class baseline. The `scale_pos_weight` parameter ensures the model pays attention to the minority fraud class despite representing less than 0.2% of training samples.

Feature importance analysis from XGBoost's gain metric reveals that `errorBalanceOrig` and `errorBalanceDest` — the engineered balance discrepancy features — are among the most predictive features. This aligns with domain knowledge: in fraudulent transactions, money is moved in ways that leave balance inconsistencies. The transaction type features (`TRANSFER` and `CASH_OUT` indicators) also rank highly, consistent with the EDA finding that fraud occurs exclusively in these transaction types.

8. Visualization / Dashboard

8.1 Web Application Overview

The project includes a fully deployed interactive web application accessible at <https://frauddetect-1gju.onrender.com/>. The frontend communicates with the backend REST API at <https://frauddetect-backend-jpgf.onrender.com/> to deliver real-time fraud predictions.

8.2 Frontend Interface

The frontend is a lightweight static web application built with plain HTML5, CSS3, and vanilla JavaScript — requiring no build step or framework. It consists of two main pages:

- Detect Page (index.html): Users enter transaction details (type, amount, origin and destination balances) and submit for an instant fraud prediction. The result is displayed with the predicted label (Fraud / Legitimate) and the fraud probability score.
- Artifacts Page (artifacts.html): Displays MLflow experiment runs, tracked metrics, parameters, and model artifacts fetched from the backend /artifacts/ endpoint — enabling visibility into experiment history.

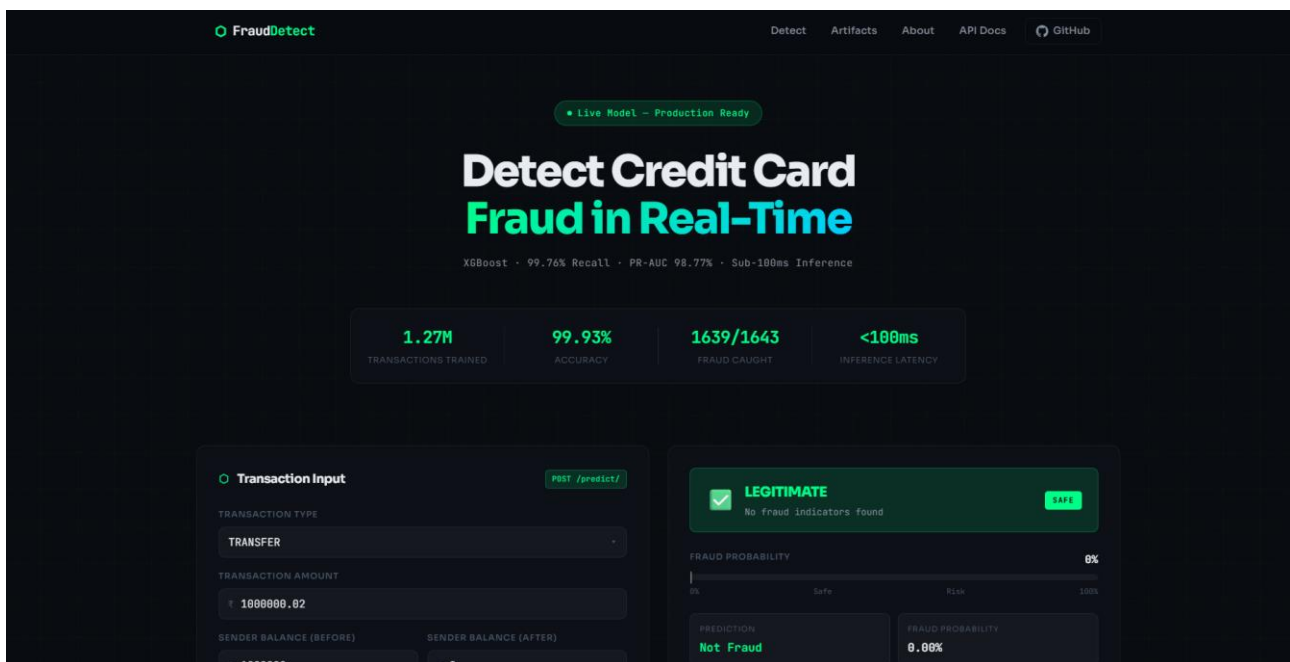
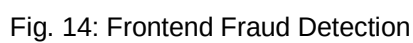
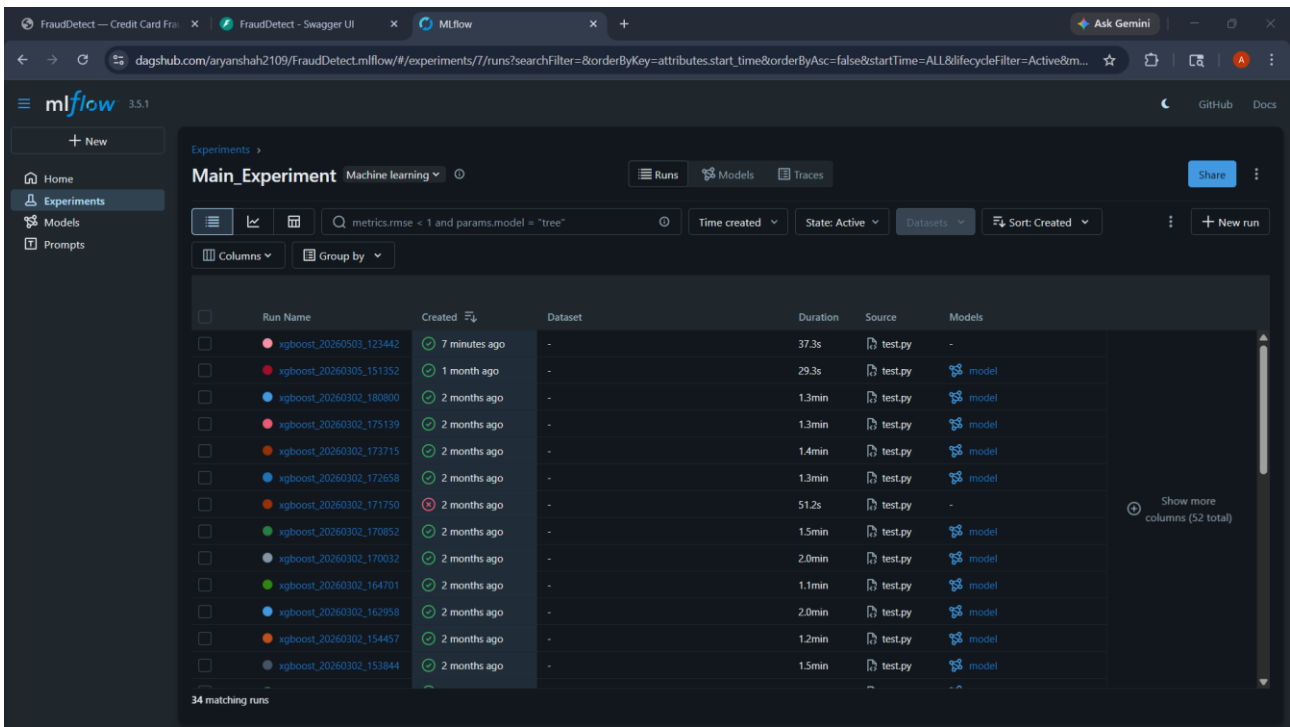


Fig. 12: Frontend UI





The screenshot displays the MLflow Experiments page for 'Main_Experiment' in a 'Machine learning' context. The interface includes a sidebar with navigation options (Home, Experiments, Models, Prompts) and a top navigation bar with tabs for Runs, Models, and Traces. A search bar and filters (Time created, State: Active, Datasets, Sort: Created) are present. The main table lists 34 matching runs, each with a Run Name, Created timestamp, Dataset, Duration, Source, and Models. The runs are sorted by creation time, with the most recent at the top. A 'Show more columns (52 total)' button is visible on the right side of the table.

Run Name	Created	Dataset	Duration	Source	Models
xgboost_20260302_123442	7 minutes ago	-	37.3s	test.py	-
xgboost_20260302_151352	1 month ago	-	29.3s	test.py	model
xgboost_20260302_180800	2 months ago	-	1.3min	test.py	model
xgboost_20260302_175139	2 months ago	-	1.3min	test.py	model
xgboost_20260302_173715	2 months ago	-	1.4min	test.py	model
xgboost_20260302_172658	2 months ago	-	1.3min	test.py	model
xgboost_20260302_171750	2 months ago	-	51.2s	test.py	-
xgboost_20260302_170852	2 months ago	-	1.5min	test.py	model
xgboost_20260302_170032	2 months ago	-	2.0min	test.py	model
xgboost_20260302_164701	2 months ago	-	1.1min	test.py	model
xgboost_20260302_162958	2 months ago	-	2.0min	test.py	model
xgboost_20260302_154457	2 months ago	-	1.2min	test.py	model
xgboost_20260302_153844	2 months ago	-	1.5min	test.py	model

Fig. 15: MLflow Experiment Runs

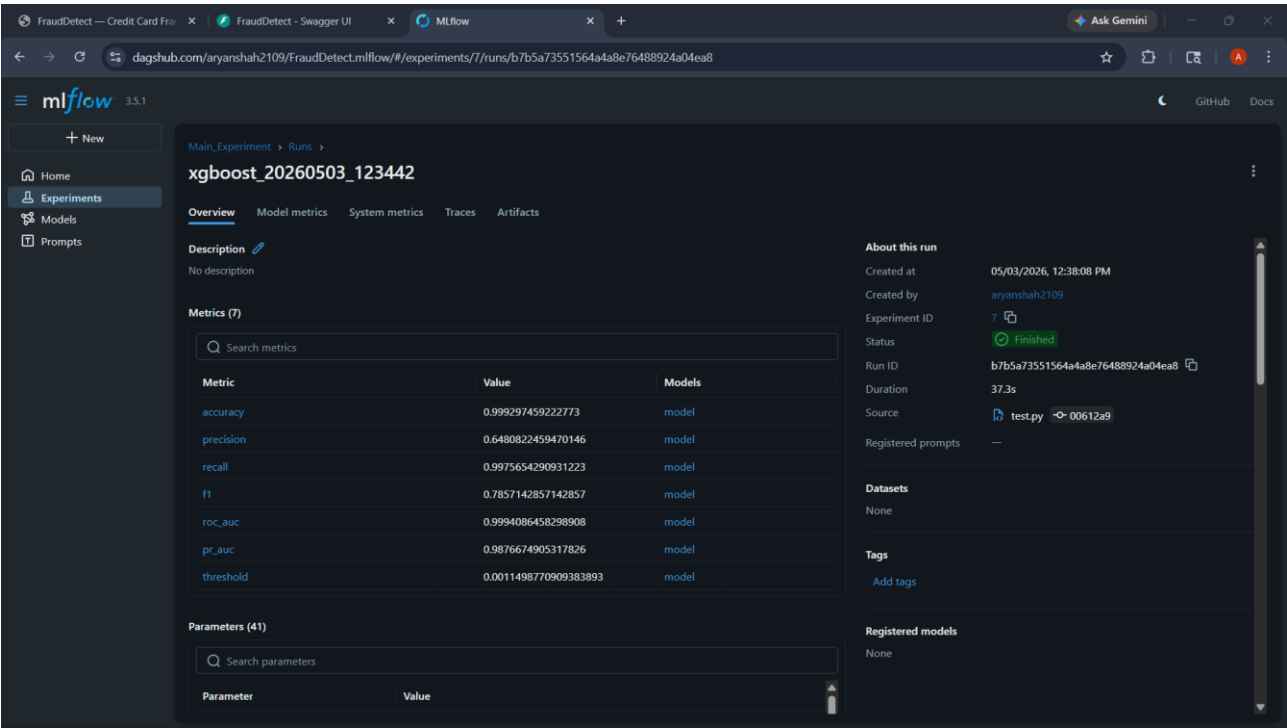


Fig. 16: MLflow latest run tracking

8.3 Backend API Documentation

The FastAPI backend exposes interactive Swagger documentation at <https://frauddetect-backend-jpgf.onrender.com/docs/>, where all API endpoints can be explored and tested.

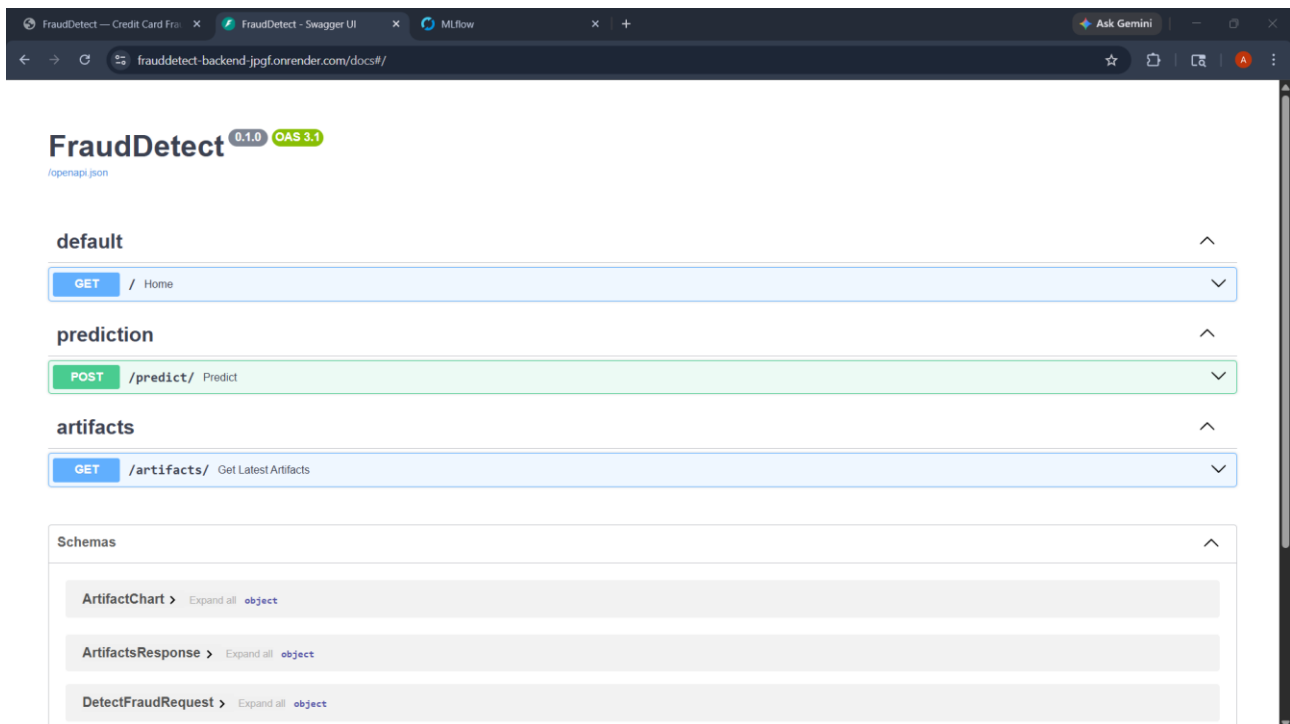


Fig. 17: FastAPI backend Documentation

8.4 Tools Used for Visualization

Tool / Library	Purpose
Matplotlib	EDA charts, training loss curves, confusion matrix heatmap
Seaborn	Correlation heatmaps, distribution plots, count plots
XGBoost plot_importance	Feature importance bar charts (gain, weight, cover)
scikit-learn	ROC and Precision-Recall curve plots
MLflow UI	Experiment comparison dashboards (loss, AUC, metrics over runs)
Vanilla JS / HTML/CSS	Interactive web dashboard for real-time predictions and artifact viewer

9. Tools and Technologies Used

Category	Tool / Library	Version	Purpose
Language	Python	3.11.9	Core programming language for ML and API
API Framework	FastAPI	0.128.0	High-performance REST API with async support
ASGI Server	Uvicorn	0.40.0	Production-grade ASGI server for FastAPI
ML Model	XGBoost	3.1.3	Gradient-boosting classifier with GPU support
Preprocessing	scikit-learn	1.8.0	StandardScaler, train-test split, metrics
Data Handling	Pandas	2.3.3	Data loading, manipulation, feature engineering
Numerical Computing	NumPy	2.4.2	Array operations and mathematical computations
Experiment Tracking	MLflow	3.9.0	Run tracking, parameter/metric logging, model registry
Data Versioning	DVC	—	Data and pipeline versioning for reproducibility
Remote Tracking	DagHub	—	Remote MLflow server and dataset hosting
Model Serialization	joblib	1.5.3	Saving and loading trained models and preprocessors
Data Validation	Pydantic	2.12.5	API request/response schema validation
Visualization	Matplotlib	3.10.8	Plotting for EDA and model evaluation
Visualization	Seaborn	0.13.2	Statistical visualization (heatmaps, distributions)
Development Env.	Jupyter Notebook	—	Interactive data exploration and EDA notebooks
Containerization	Docker / Docker Compose	—	Containerized deployment for consistent environments
Cloud Deployment	Render	—	Cloud hosting for both frontend and backend
Frontend	HTML5 / CSS3 / JS	—	Static web UI for real-time predictions and artifact viewer

10. Conclusion

This project successfully delivers FraudDetect — a production-ready, end-to-end credit card fraud detection system that combines rigorous machine learning engineering with modern MLOps practices and a deployed cloud web application.

10.1 Summary of Findings

- An XGBoost classifier with 263 estimators, precision-constrained threshold optimization, and domain-engineered features (balance error) achieves strong discriminative performance on a highly imbalanced transaction dataset with over 6 million records.
- The engineered balance-error features (errorBalanceOrig, errorBalanceDest) are among the most predictive signals, confirming that fraudulent transactions leave detectable anomalies in account balance changes.
- Fraud occurs exclusively in TRANSFER and CASH_OUT transaction types in this dataset, a pattern that the model successfully learns and leverages.
- The precision-recall threshold optimization strategy effectively balances the business trade-off between catching more fraud (recall) and minimizing false alarms for legitimate customers (precision $\geq 65\%$).
- The FastAPI microservice delivers predictions in under 100ms, meeting real-time integration requirements.

10.2 Objectives Achieved

- End-to-end ML pipeline: Data ingestion, preprocessing, feature engineering, model training, evaluation, and serialization — all implemented in a modular, reproducible pipeline.
- Production-grade API: FastAPI REST service with Pydantic validation, global error handling, CORS support, singleton model loading, and comprehensive logging.
- MLOps integration: MLflow experiment tracking and DVC data versioning ensure every training run is recorded and reproducible.
- Cloud deployment: Both frontend (<https://frauddetect-1gju.onrender.com/>) and backend (<https://frauddetect-backend-jpgf.onrender.com/>) are live and accessible.

11. Future Scope

11.1 Algorithmic Improvements

- Investigate deep learning approaches such as TabNet or transformer-based models for tabular data, which may capture higher-order feature interactions.
- Explore graph neural networks (GNNs) to model relationships between accounts and detect organized fraud rings.
- Implement online learning or continual learning strategies so the model can adapt to evolving fraud patterns without full retraining.
- Experiment with SMOTE, ADASYN, or other oversampling strategies during training to further address class imbalance.

11.2 System & Infrastructure Improvements

- Add authentication (JWT / OAuth2) and rate limiting to the API for production security.
- Implement a streaming pipeline (Apache Kafka + Flink) for true event-driven, real-time fraud scoring at scale.
- Add a model monitoring dashboard (e.g., Evidently AI or Grafana) to detect data drift and model performance degradation in production.
- Build an automated retraining pipeline triggered by detected drift, with MLflow model registry managing promotion from staging to production.

11.3 Frontend & User Experience

- Add batch prediction support — allow uploading a CSV of transactions for bulk fraud screening.
- Build a real-time monitoring dashboard displaying live transaction throughput, fraud rate trends, and model confidence distributions using D3.js or Chart.js.
- Add explainability outputs (SHAP values) in the API response so analysts understand why a specific transaction was flagged.

12. References

Dataset

1. Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. Proceedings of the 28th European Modelling and Simulation Symposium.
2. Kaggle Dataset: "Synthetic Financial Datasets For Fraud Detection" — <https://www.kaggle.com/datasets/ealaxi/paysim1>

Research Papers

3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
4. Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating Probability with Undersampling for Unbalanced Classification. IEEE SSCI 2015.
5. Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. Decision Support Systems.
6. Sculley, D., Holt, G., Golovin, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. NIPS 2015.

Documentation & Frameworks

7. FastAPI Documentation — <https://fastapi.tiangolo.com/>
8. XGBoost Documentation — <https://xgboost.readthedocs.io/>
9. MLflow Documentation — <https://mlflow.org/docs/latest/>
10. scikit-learn Documentation — <https://scikit-learn.org/stable/>
11. DVC Documentation — <https://dvc.org/doc>

Books

12. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.
13. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

Appendix

A. Project Repository & Deployment

Resource	URL / Reference
GitHub Repository	https://github.com/aryanshah2109/FraudDetect
Frontend (Deployed)	https://frauddetect-1gju.onrender.com/
Backend API (Deployed)	https://frauddetect-backend-jpgf.onrender.com/
API Documentation	https://frauddetect-backend-jpgf.onrender.com/docs

B. Sample API Request and Response

Endpoint: POST /predict/

Request Body:

```
{
  "type": "TRANSFER",
  "amount": 1000000,
  "oldbalanceOrg": 1000000,
  "newbalanceOrig": 0,
  "oldbalanceDest": 0,
  "newbalanceDest": 1000000
}
```

Response:

```
{
  "prediction": 1,
  "prediction_label": "Fraud",
  "fraud_probability": 0.8537533283233643,
  "risk_factors": [
    "Sender balance equals transaction amount",
    "Sender balance drained to zero",
    "Transaction type commonly used in fraud"
  ],
  "threshold": 0.0011498770909383893
}
```

C. Project File Structure

Key directories and files:

- **artifacts/** — Stores experiment outputs organized by runs (metrics, plots, configs, predictions)
- **data/** — Dataset storage (raw, processed, and intermediate files used for training/inference)
- **docs/** — Project documentation, reports, and design references
- **frontend/** — UI layer (HTML/CSS/JS or framework-based frontend for visualization and interaction)
- **logs/** — Logging outputs for training, inference, and application debugging
- **models/** — Saved ML models, preprocessors, and possibly versioned checkpoints
- **notebooks/** — Jupyter notebooks for EDA, experimentation, and prototyping
- **references/** — External resources, research papers, or supporting materials
- **reports/** — Generated reports such as evaluation summaries, visualizations, or PDFs
- **src/** — Core source code (data processing, feature engineering, training pipelines, inference logic)
- **tests/** — Unit and integration tests for validating functionality
- **venv/** — Local Python virtual environment (should not be committed)

Configuration & setup files

- **.env** — Environment variables (API keys, configs, secrets)
- **.gitignore** — Specifies files/folders to exclude from version control
- **.dockerignore** — Files excluded from Docker builds
- **docker-compose.yml** — Multi-container setup for local development
- **Dockerfile** — Defines container image for the application
- **Makefile** — Shortcut commands for build, run, test, etc.
- **requirements.txt** — Production Python dependencies
- **requirements-dev.txt** — Development dependencies (testing, linting, etc.)
- **setup.py** — Package configuration for project installation

Project & metadata

- **README.md** — Project overview, setup instructions, and usage guide
- **LICENSE** — Licensing information
- **src.egg-info/** — Metadata generated when packaging/installing the project
- **.dvcignore** — Ignore rules for DVC (if using data versioning)
- **.project-root** — Marks root directory for tooling/scripts
- **tox.ini** — Configuration for automated testing across environments

Utility / misc files

- **test_environment.py** — Script to validate environment setup
- **test.py** — Run re-training pipeline

D. Assessment Rubrics

Sr. No.	Criteria	Max Marks	Marks Obtained
1	Problem Definition & Objective Clarity	1	
2	Dataset Understanding & Description	1	
3	Data Preprocessing & EDA	2	
4	Model/Algorithm Implementation	2	
5	Result Analysis & Interpretation	1	
6	Visualization / Dashboard	1	
7	Documentation & Report Quality	1	
8	Presentation / Viva	1	
Total		10	

Date

Signature